

2. Phase 0 – Scope & success criteria (3–4 hours)

Goal: make the project small enough to finish, big enough to be impressive.

2.1 Pick ONE universal workflow

For MVP, choose one painful, repeatable workflow that works in any company that uses email + a CRM:

Example: “After a sales meeting, auto-log to CRM + draft follow-up email + create tasks.”

Why this one:

- Universal (any sales team).
- Easy to explain.
- Easy to measure (time saved, CRM completeness, follow-up rate).

2.2 Pick ONE CRM + ONE email provider

- CRM: HubSpot or Salesforce (pick whichever you can get test access to; HubSpot is easier for MVP).
- Email: Google Workspace first (Gmail + Calendar + Drive); add Outlook later.

2.3 Define success metrics

For the chosen workflow, define:

- Primary metric: e.g. “Time from meeting end → CRM updated + follow-up sent”.
- Secondary metrics:
 - % of meetings where agent is used.
 - % of agent drafts accepted without edit.
 - CRM field completeness (e.g. notes, next_step, deal_stage).
 - Error rate / retry rate.

Write these down; you’ll use them in evaluation.

2.4 Define “MVP done” checklist

MVP is done when all of these are true:

- User can connect:
 - Google account (read email + calendar),
 - CRM account (HubSpot/Salesforce),
 - Optional: Google Drive / OneDrive.
 - For any recent meeting (last 7 days), user can:
 - See the meeting in your UI.
 - Trigger the “Post-meeting workflow”.
 - See CRM note + follow-up email + tasks created.
 - User can edit email before sending.
 - Every agent run is traced; you can see:
 - Steps, tools called, latency, token cost, errors.
 - At least 3 pilot users (friends/former colleagues) have:
 - Used it on real work.
 - Filled a short feedback form.
 - You have at least 1 workflow that runs end-to-end with <5% error rate over 20 runs.
-

3. Phase 1 – Design system & contracts (6–8 hours)

Goal: decide architecture, data model, and APIs before writing any code.

3.1 Architecture

Use a hybrid workflow + agent pattern:

- Deterministic workflow: handles routing, validation, retries.
- LLM agent steps: only where reasoning is needed (summarization, drafting, planning).
- Tool layer: separate functions for each external system (email, CRM, docs).

Recommended structure:

- `api/` – REST API (auth, tenants, workflows, runs).
- `worker/` – background job runner (trigger workflows).
- `agent/` – LLM orchestration (chains/tools).
- `connectors/` – email, CRM, docs adapters.
- `observability/` – tracing, logging, metrics.
- `web/` – frontend SPA.

3.2 Core data model

Define schemas for:

- `Tenant` (company/workspace)
- `User` (with role: admin/member)
- `Connection` (oauth tokens for Google, CRM, etc.)
- `Workflow` (e.g. “post_meeting_followup”)
- `WorkflowRun` (one execution of a workflow)
- `WorkflowStep` (individual step: input, output, tool calls, status)
- `Message` (emails, chat messages)
- `Artifact` (CRM notes, tasks, documents created by agent)

Enforce:

- Every `WorkflowRun` belongs to a `Tenant` and `User`.
- Every `WorkflowStep` has `parent_run_id`, `tool_name`, `latency_ms`, `tokens_used`, `status`.
- All timestamps in UTC; store timezone offset separately.

3.3 API contracts

Define these endpoints early:

- **Auth & tenants**
 - `POST /auth/login`
 - `POST /auth/register`
 - `GET /tenant`
- **Connections**
 - `GET /connections` (list)
 - `POST /connections/google` (start OAuth)

- `POST /connections/crm`
- **Workflows**
 - `GET /workflows`
 - `POST /workflows/{id}/runs` – trigger a run
 - `GET /workflows/runs/{id}` – status + trace
 - `GET /workflows/runs/{id}/steps`
- **Artifacts**
 - `GET /artifacts` – list created notes/emails/tasks
 - `GET /artifacts/{id}`

Keep the API small and stable; you can extend later.

4. Phase 2 – Core agent runtime (12–15 hours)

Goal: a solid, production-like agent loop that you can reuse across workflows.

4.1 Choose your stack

- Language: TypeScript (Node) or Python – pick whichever you’re faster with.
- Agent framework: optional but helpful (LangChain, Mastra, etc.), or just raw OpenAI/Anthropic SDK + your own orchestrator.
- DB: Postgres (via Prisma/Drizzle/SQLAlchemy).
- Queue: Redis + BullMQ (or similar) for workflow runs.
- Observability: Langfuse / Helicone / Braintrust or roll your own with OpenTelemetry + Postgres.

4.2 Implement the workflow engine

Build a simple graph executor:

- A `Workflow` is a list of `Step` objects.
- Each `Step`:
 - Has a type: `llm`, `tool`, `parallel`, `branch`.
 - Declares input/output schema.
 - Optionally has a `timeout_ms`, `retry_policy`.

- Execution:
 - Persist `WorkflowRun` and `WorkflowStep` rows.
 - On each step:
 - Record start time.
 - Call step handler (LLM or tool).
 - Record end time, output, error, tokens.
 - On error:
 - Mark step as failed.
 - Decide: retry, fail run, or pause and notify user.

4.3 Implement the LLM step

For each LLM step:

- Accept:
 - `system_prompt`, `user_prompt`, `tools`, `output_schema`.
- Do:
 - Call model (with tools if provided).
 - If tool calls: execute them and feed results back.
 - Continue until final answer or max iterations.
- Enforce:
 - `max_iterations` (e.g. 3) to avoid infinite loops.
 - `timeout_ms` per step.
 - Separate “reasoning” vs “tool execution” clearly.

4.4 Implement tool layer

Tools are pure, deterministic functions with clear contracts:

Example tools:

- `search_recent_emails(query, limit)`
- `get_calendar_event(event_id)`
- `get_crm_contact(contact_id)`
- `create_crm_note(contact_id, body)`
- `create_crm_task(contact_id, title, due_date)`
- `send_email(to, subject, body)`

Design rules:

- Each tool does one thing.
 - Input/output are strongly typed (JSON Schema).
 - Tools must be idempotent where possible (e.g. same email not created twice).
 - Tools must log:
 - External system called.
 - Latency.
 - Whether call succeeded.
-

5. Phase 3 – Connectors: email, CRM, docs (15–18 hours)

Goal: make the agent actually useful by wiring real data.

5.1 Email connector (Google Workspace)

Implement:

- OAuth flow for `gmail.readonly`, `calendar.readonly`, `drive.readonly`.
- On connection:
 - Sync last 30 days of:
 - Emails (sent/received).
 - Calendar events (attendees, description).
 - Store in your DB as `Message` and `Event` (linked to Tenant/User).
- Webhooks:
 - Use Google push notifications or polling for new emails/events.
- Expose tools:
 - `search_emails(query, from, to, after, before)`
 - `get_email_thread(thread_id)`
 - `get_calendar_event(event_id)`

Security:

- Store tokens encrypted at rest.
- Never log raw email content; log only metadata + hashes if needed.

5.2 CRM connector (HubSpot or Salesforce)

Implement:

- OAuth for CRM.
- Sync:
 - Contacts, companies, deals, tasks.
- Expose tools:
 - `get_contact_by_email(email)`
 - `create_note_for_contact(contact_id, body)`
 - `create_task_for_contact(contact_id, title, due_date)`
 - `update_deal_stage(deal_id, stage)`

Idempotency:

- Use CRM's native idempotency keys or your own `external_id` on artifacts to avoid duplicates.

5.3 Docs connector (optional for MVP)

If time:

- Google Drive / OneDrive:
 - `search_documents(query)`
 - `read_document(doc_id)` (first N pages).
- Use RAG only if you need large corpora; for MVP, simple keyword search is fine.

6. Phase 4 – Observability & evaluation (10–12 hours)

This is what separates “cool prototype” from “reliable product”. Observability + evals are mandatory, not optional.

6.1 Tracing

For every `WorkflowRun` and `WorkflowStep`, ensure you log:

- Run: tenant, user, workflow_type, status, total_duration, total_tokens.
- Step: step_type, tool_name, input_summary (not full PII), output_summary, latency_ms, tokens, error.

Use either:

- A hosted tool (Langfuse, Helicone, Braintrust), or
- Your own tables + OpenTelemetry spans.

6.2 Metrics

Track and expose:

- Runs per workflow per day.
- Success rate per workflow.
- P50/P90 latency per workflow.
- Token cost per workflow.
- Tool error rates.

6.3 Evaluation suite

Create a small but growing eval dataset:

- Manually craft 10–20 “golden scenarios”:
 - Example: “Meeting with X about Y; CRM contact exists; no prior follow-up.”
- For each:
 - Define expected behavior:
 - CRM note should contain X, Y.
 - Email should be polite, <200 words, mention Z.
 - Task due date should be 24–48h.
- Run your agent on these scenarios:
 - Use an LLM-as-judge or simple rule checks.
 - Compute pass/fail.

You’ll run this eval on every change to prompts or logic.

7. Phase 5 – UX & admin dashboard (15–18 hours)

Goal: make the product feel real and trustworthy.

7.1 Minimal UI pages

- `/login` – email/password or Google SSO.
- `/onboarding`
 - Connect Google.
 - Connect CRM.
 - Pick default workflow.
- `/dashboard`
 - Recent runs (status, time, workflow type).
 - Quick “Run workflow” button for recent meetings.
- `/workflows`
 - List workflows.
 - Enable/disable.
- `/runs/{id}`
 - Step-by-step trace.
 - Artifacts created (link to CRM note, email, task).
- `/settings`
 - Connections, API keys, billing.

7.2 Workflow configuration

For the chosen workflow (e.g. `post_meeting_followup`), allow:

- Toggle: “Auto-run after meetings” vs “Manual only”.
- Settings:
 - Default follow-up delay (e.g. 1 hour).
 - Email template (tone, length, disclaimers).
 - CRM note format.

7.3 Trust & transparency

Show users:

- What the agent will do before doing it.
- After the run:
 - Which emails were read.
 - Which CRM records were updated.
 - The exact email/task that was created.

Let them:

- Edit email before sending.
 - Revert or modify CRM changes.
-

8. Phase 6 – Hardening & launch (15–20 hours)

Goal: make it safe and stable for real users.

8.1 Security & privacy

- Encrypt all OAuth tokens at rest.
- Never log raw email bodies or CRM content; log only identifiers.
- Row-level security: every query filtered by `tenant_id`.
- Rate limits per tenant per workflow.
- Audit log for sensitive actions (e.g. “send_email”, “update_crm”).

8.2 Error handling & retries

- For each tool call:
 - Distinguish retryable (5xx, rate limit) vs non-retryable (4xx).
 - Implement retry with exponential backoff for retryable errors.
- For workflows:
 - Mark as “failed” after max retries.
 - Show clear error in UI and allow re-run.

8.3 Idempotency & deduplication

- Use `idempotency_key` on workflow runs and artifacts.
- Prevent the same meeting from triggering multiple runs.
- Prevent creating the same CRM note or email twice.

8.4 Testing

- Unit tests:
 - Tools (mocked external APIs).
 - Workflow engine (step execution, retries).
- Integration tests:
 - Full workflow with test Google/CRM accounts.
- Load tests:
 - Simulate 10–20 concurrent runs; check latency and error rate.

8.5 Deployment

- Use a managed platform (Render, Fly.io, Railway, Vercel + separate worker).
 - Set:
 - CI/CD (GitHub Actions).
 - Staging environment.
 - DB backups.
 - Monitoring:
 - Uptime checks.
 - Alert on error spikes or queue backlogs.
-

9. Phase 7 – Post-launch iteration (ongoing)

Once you have real users:

1. Add workflows:
 - “Follow up on stalled deal.”
 - “Summarize email thread and create tasks.”
 - “Turn support email into CRM ticket + reply draft.”
2. Add more CRMs / email providers:
 - Outlook/Exchange.

- Pipedrive, Close, etc.
 - 3. Improve evaluation:
 - Grow eval dataset from user anonymized logs.
 - Add automated regression tests.
 - 4. Optimize costs:
 - Cache similar LLM calls.
 - Use cheaper models for simple steps.
-

10. Checklist: “Will this ship 100%?”

Use this as a final filter. If any item is missing, you’re not done.

Scope & Design

- One clearly defined universal workflow (e.g. post-meeting follow-up).
- One CRM + one email provider chosen.
- Success metrics defined and measurable.
- MVP “done” criteria explicitly listed and met.

Architecture & Data

- Architecture diagram: API, worker, agent, connectors, observability.
- Data model defined (Tenant, User, Connection, Workflow, Run, Step, Artifact).
- API contracts documented and stable.
- All timestamps in UTC; all entities have `tenant_id`.

Agent & Tools

- Workflow engine with steps (LLM, tool, parallel, branch).
- LLM step with max iterations, timeouts, and separation of reasoning vs tools.
- Tool layer: each tool does one thing, with typed input/output, idempotency, and logging.
- Connectors for email, CRM, docs implemented with OAuth and sync.

Observability & Evaluation

- Every run and step is traced (latency, tokens, tool calls, errors).
- Key metrics tracked (runs, success rate, latency, cost).
- Eval dataset with 10–20 golden scenarios and automated checks.

UX & Trust

- UI pages: login, onboarding, dashboard, workflows, runs, settings.
- Users can see what the agent will do and what it did.
- Users can edit emails and revert/modify CRM changes.
- Workflow settings (auto/manual, templates) configurable.

Hardening & Launch

- Security: encryption at rest, row-level security, rate limits, audit log.
- Error handling: retry vs fail, clear errors in UI.
- Idempotency: no duplicate runs or artifacts.
- Testing: unit, integration, basic load tests.
- Deployment: staging, CI/CD, monitoring, backups.

If you follow this playbook, you won't just ship "something" — you'll ship a reliable, observable, monetizable AI workflow product that actually works in the real world and is positioned to grow into a serious business.

. High-level roadmap